

Design and Evaluation of a Microservice Architecture for a Fractional Real Estate Investment Platform

Hassan Ishfaq Hussain

Faculty of Computing, Blekinge Institute of Technology, SE-371 79 Karlskrona Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the Diploma in Software Engineering. The thesis is equivalent to 10 weeks of full-time studies.

Contact Information:
Author(s): Hassan Ishfaq Hussain
Firstname: Hassan Ishfaq
E-mail: hassanihussain1998@gmail.com

University advisor:
Sadi Alawadi
Department of Computer Science

Faculty of Computing
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet: www.bth.se
Phone: +46 455 38 50 00
Fax: +46 455 38 50 57

Abstract

This thesis designs and evaluates a microservice-based architecture for a fractional real estate investment platform within the fintech domain. The study addresses limitations of monolithic systems in scalability, modularity, and maintainability by proposing a distributed system composed of independent services coordinated through an API Gateway. A design science approach is used, where the architecture is implemented as a minimum viable product and evaluated through empirical experiments.

The evaluation compares key architectural decisions, including local authentication versus OAuth 2.0, static JSON versus API-based data access, client-side rendering performance, and system scalability under load. Results show clear trade-offs between performance and architectural flexibility. The proposed microservice architecture improves modularity and scalability but introduces communication overhead and increased complexity.

The study concludes that microservice architectures are a suitable architectural approach for fintech systems when service boundaries and communication patterns are carefully designed, based on the observed behaviour in the implemented MVP and experimental environment.

Keywords: microservices, architecture design, fractional real estate platform, fintech application

Acknowledgments

I would like to extend my gratitude to all the teachers that have collaborated on dbwebb portal for all the instructions and resources that they have assembled and shared via that medium, they have helped me greatly during my tenure at Blekinge Technical University.

Also I would like to take this opportunity to thank all my family members who have supported me and understood my unavailability. On a handful of days I had to focus more on my studies and they have accommodated my learning routines and habits during those occurrences.

I would also like to thank the co-founder, Ibrahim Sohail Dar at Earnsome, a startup that has helped me work on this thesis for the betterment of their platform's initial developmental decisions.

Contents

Abstract **1**
Acknowledgments **2**
Chapter 1 **5**
Introduction 5
1.1 Background 5
1.2 Motivation and Value 6
1.3 Purpose 7
1.4 Scope 7
1.5 Limitations 7
1.6 Previous research 8
Chapter 2 **9**
Research Questions 9
2.1 Focus areas 9
2.1.1 Areas of analysis 9
2.2 Research Questions 9
2.2.1 Research Question 1 9
2.2.2 Research Question 2 9
2.2.3 Research Question 3 10
Chapter 3 **11**
Methods 11
3.1 Literature Review 11
3.2 Experimental Environment and System Setup 13
3.3 Experimental Procedure 14
3.4 Experiments 15
Chapter 4 **18**
Architecture Design 18
4.1 Architectural Foundations 18
4.2 Platform Workflow Overview 20
4.3 Service decomposition 21
4.4 Proposed System Architecture 22
4.5 Authentication Architecture Design 23
4.6 Data Architecture Design 25
4.7 Communication Architecture 25
Chapter 5 **27**
Results and Evaluation 27
5.1 Experiment 1 - Authentication Architecture: Local Auth vs OAuth2.0 27
5.2 Experiment 2- Data Access: Static JSON vs API and Database 28

5.3 Experiment 3 - Frontend Rendering Performance 29
5.4 Experiment 4 – Load Testing Results 30
Chapter 6 **31**
Analysis and Discussion 31
6.1 Research Question 1 31
6.2 Research Question 2 32
6.3 Research Question 3 34
6.4 Overall Discussion 36
Chapter 7 **37**
Conclusion 37
Chapter 8 **39**
Future Work 39
References **40**
Attachment A - WireShark Results 42
A1: Static JSON API Result 42
A2: MongoDB API Result 42
Attachment B - GitHub repositories 43

Chapter 1

Introduction

This paper shall contribute by proposing an architectural design that can be used to create the core architecture for a fractional real estate investing platform, which is based on the idea of fractional ownership, where any investor can purchase a portion or share of an asset, that gives them the right to equal portion of returns from that asset, in this case a commercial real estate asset [1].

Moreover a microservice architecture will be proposed based on the needs for such an investment platform and few of the decisions made inside the decomposed services based on the businesses need will be tested and evaluated in this study [2].

The structure of this paper follows the development and evaluation process of the proposed architecture. First, the research questions and experimental setup are described in Chapter 2 and 3, giving an overview of the study and experiments that will be conducted on proposed architecture-based services. This is followed by a description of the proposed architecture and its service decomposition, in Chapter 4. Then the results from the experiments are put forward in Chapter 5. Finally, Analysis of the results is carried out in Chapter 6, followed by key conclusions derived in Chapter 7. Ending the study with suggestions on future improvements in Chapter 8.

1.1 Background

Digital investment platforms based on a fractional ownership approach have transformed physical financial markets by enabling broader public access to assets that were previously limited to institutional or high-net-worth investors. This gives the spending power to the initial owner of the asset, whose money is not tied down to a single project and can focus on expanding their business due to the shared costs and responsibilities that trickle down to the pool of co-owners. This provides the co-owners lower financial commitment, access to higher-value assets and potential for appreciation and an income from their investments [1].

The need for such a digital investment platform arises in the market creating a gap that can be filled by a platform that is scalable and modular, which can have loosely coupled services that can easily be customized to different target groups, business and market needs. Here, the scalability and modularity of a microservice architecture becomes relevant, that is in comparison much similar to a well-known architectural pattern that is commonly implemented at an enterprise-level i.e; Service-Oriented Architecture (SOA) [3] [4].

In our case a microservice architecture is implemented with a similar approach at an application-level because of its architectural flexibility, and as an optimal solution for the business domain requirements of an investment platform [4] [5].

Microservice architecture based real estate investment platform is introduced as this architecture is gaining traction and is one of the trending and upcoming architectural choices for the modern web technological decisions [5].

While microservice architecture offers many advantages it also introduces an increased system complexity where an application is decomposed into distributed systems that are loosely coupled to form an advanced modular, service oriented and flexible system, whereas building such platforms introduce architectural challenges related to scalability, security, data, and maintainability [6] [7].

This thesis explores and proposes the microservice architectural design for a scalable fractional real estate investment platform. The platform will be implemented as a Minimum viable product (MVP) to validate architectural decisions, but the primary focus of the thesis lies in experimenting, analyzing and evaluating behind-the-scenes architectural decisions and technological choices.

1.2 Motivation and Value

The motivation for this study arises from increased demand for scalable and flexible digital investment platforms solutions, particularly in the domain of fractional real estate ownership. Traditional monolithic financial applications face significant limitations in scalability, maintainability and adaptability, making them unsuitable for the evolving landscape of modern financial technology environments that require customization flexibility and growth of such technological platforms.

Fractional real estate investment introduces additional complexity, as a single asset is shared among multiple investors, requiring reliable data handling, secure authentication and efficient system scalability. These requirements place significant pressure on traditional system architectures, which can lead to performance bottlenecks and difficulties in system maintainability and evolution.

The need to explore how a microservice-based architectural approach can address these challenges has motivated this study. Although microservice architectures have been widely studied, limited research provides practical architectural guidance for fractional real estate investment platforms. This creates a need for research that bridges general microservice theory with the specific requirements of fractional ownership fintech systems. The desired outcome will be achieved through the decomposition of a system into loosely coupled services leading to a proposal of an architectural design that is scalable, secure, maintainable and modular. This is particularly relevant for fintech applications where business services' flexibility is of paramount importance.

In addition to business specific functional and technical challenges, there is an intention to design and prototype a technical foundation that could support and influence a startup initiative in the fintech domain.

Finally, the academic value of this study is the exploration and design for proposing a microservice architecture based on the needs of a financial application. And the practical value of this study lies in the experiments carried out demonstrating the proposed architectural design while indirectly contributing to the increase in accessibility for the common man into high stake investment projects.

1.3 Purpose

The main aim of this study is to design and propose an architecture and evaluate the architectural decisions that can support a scalable fractional investment platform for both web and mobile clients through shared backend services.

From an academic standpoint, the study enhances understanding of microservice architectural decisions factors in emerging fintech systems. From a professional standpoint, it develops skills in system architecture, scalability planning, and secure full-stack design. While the thesis focuses on selected architectural decisions, analysis, technology considerations, and the development of an MVP, the insights gained will form the foundation for a future production-scale implementation of the platform. The proposed architecture is intended to serve as a base for a fully scalable and customisable fintech system.

This will allow the envisioned fintech startup solution to scale the system for real-world fractional real estate investment and potentially support expansion into additional markets and use cases after the thesis is completed

1.4 Scope

This study is limited to the creation of the core of a fractional real estate application that is limited to a minimum viable product (MVP). It is based on the microservice architecture that will be designed, proposed and implemented during the timeframe of this work.

Due to the scope of a 15-credit university diploma thesis, the thesis focuses on proposing a microservice architecture, supported by the decomposition of core services for the platform. A limited set of functionalities from these services will be selected for evaluation and analysis.

The application will be designed as a scalable, modular and modern system with the intention of enabling future expansion into a fully functional fintech application.

1.5 Limitations

This study excludes the payment, transactional, messaging, business, legal and financial aspects of the platform. The proposed architecture will be implemented only to the extent required to support the experiments and validate the core architectural decisions.

The system is implemented as a minimum viable product (MVP), meaning that only selected parts of the full platform are developed to support experimental evaluation. As a result, the

study does not represent a complete production-ready system but rather a simplified implementation used to investigate architectural trade-offs.

1.6 Previous research

Literature relevant to this study covers several key areas, including microservice architecture principles, assessment of performance, modularity and scalability, use cases in fintech, service granularity challenges in distributed systems, and architectural design and monitoring approaches.

Growing popularity of fractional ownership models indicates a growing shift towards platform based applications for shared asset ownership, although guidance for architectural design and implementations for such systems remain limited.

This study builds on these research directions by exploring how microservice-based design can be applied and evaluated in a fractional real estate investment platform's context.

While previous studies provide extensive knowledge regarding microservice architectures, scalability, service decomposition, and fintech system design, a specific architectural framework for fractional real estate investment platforms remains largely unexplored. Existing research primarily focuses on general microservice principles or broader fintech applications rather than the architectural requirements of fractional ownership systems. Furthermore, limited empirical work evaluates architectural trade-offs such as authentication approaches, data access strategies, and scalability considerations within this context. This study addresses that gap by proposing and evaluating a microservice-based architecture tailored to a fractional real estate investment platform and assessing selected architectural decisions through controlled experiments.

Chapter 2

Research Questions

2.1 Focus areas

The thesis focuses on architectural design decisions for a secure and scalable fintech platform that enables fractional real estate investment. The emphasis lies on system-level reasoning rather than feature completeness. An MVP will be implemented to validate selected architectural choices but not included in the study, only the reference to the code repositories will be done where required and a link to the repositories is added in the appendix B.

2.1.1 Areas of analysis

- Architectural design principles for scalable fintech systems
- Security considerations in web-based investment platforms, focused solely on authentication flows within the architectural construction of the MVP
- Limited scalability strategies
- System evolution scenarios
- System design's choice evaluation

2.2 Research Questions

2.2.1 Research Question 1

How can a microservice architecture be designed to support the core functionality of a fractional real estate investment platform?

This research question aims to define how the services for a real estate investing fintech application can be decomposed into a microservice-based architecture using the separation of concerns to support the core functionality of the investment platform.

2.2.2 Research Question 2

How does architectural data and service layer design affect modularity, maintainability, and performance?

This research question aims to evaluate how different architectural decisions in data and service layers impact modularity, maintainability, performance and scalability of the system.

2.2.3 Research Question 3

How well does the proposed architecture support scalability and system evolution when evaluated using defined architectural scenarios such as increasing numbers of users and property listings ?

This research question aims to assess the scalability and robustness of the proposed architecture under increased system load and growth scenarios such as increased users and property listings.

Chapter 3

Methods

3.1 Literature Review

The collection of literature for this study is primarily conducted from academic and scientific databases provided by Blekinge Tekinsk Högskola (BTH), including BTH Library resources, Google Scholar, ResearchGate and ScienceDirect [8] [10] [11] [15]. While these platforms were used as discovery tools, the academic quality was established by prioritizing peer-reviewed journals and conference proceedings for all core claims regarding software architecture, microservices, and fintech systems.

All figures included in this study were designed by the author using Lucidchart platform, an online tool for creating diagrams and system visualizations. These visual representations are used to illustrate architectural designs, service decompositions and system workflows in a clear and structured manner.

This study follows a design science research approach for information systems, which is used to structure the investigation and answer the research questions [9]. As illustrated in Figure 1, the approach begins with identifying a problem, followed by designing a proposed solution, and finally solidifying the claims of the proposed solution through quantitative evaluation.

$$Problem \longrightarrow Solution \dashrightarrow Evaluation$$

Figure 1: Design science approach

To operationalize the Design Science Research (DSR) approach shown in Figure 1, the research process is structured into clearly defined phases based on the methodology proposed by Peffers et al. [16]. These phases translate the general DSR model into the specific context of this study, where a microservice-based architecture is designed, implemented as an MVP, and evaluated through quantitative experiments. This structured breakdown ensures traceability between the identified problem, the proposed architectural solution, and the empirical evaluation of its performance and design trade-offs.

The following table presents the adapted Design Science Research process as applied in this thesis.

Table 1. Design Science Research process applied in this thesis

Steps	Description
Problem identification and motivation	Traditional monolithic architectures in fintech and investment platforms face limitations in scalability,

	modularity, and maintainability. This thesis addresses these challenges in the context of a fractional real estate investment platform requiring distributed, service-based design.
Definition of Objectives for a Solution	The objective is to design a microservice-based architecture that enables separation of concerns, independent service deployment, and improved scalability for a fractional investment platform.
Design and development	A microservice architecture is designed consisting of an API Gateway and core services including authentication, user management, and property management services, following domain-driven decomposition principles.
Demonstration (MVP Implementation)	The proposed architecture is implemented as a Minimum Viable Product (MVP), where selected services are developed to represent the system and enable evaluation of architectural decisions under realistic conditions.
Evaluation	The MVP is evaluated through four quantitative experiments: authentication mechanisms (local vs OAuth 2.0), data access strategies (static JSON vs API/database), frontend rendering performance under different dataset sizes, and system scalability under increasing concurrent load.
Communication	The architectural design, implementation, and experimental results are documented, analyzed, and discussed throughout the thesis to answer the defined research questions.

The structured DSR process presented in Table 1 provides the methodological foundation for this study. It ensures that each stage of the research is explicitly linked, from problem identification to evaluation and communication of results. This structure supports the systematic development of the microservice-based architecture and ensures that the evaluation through experiments is directly grounded in the design decisions made during the development of the MVP. The following sections further describe the experimental setup and implementation details used to evaluate the proposed architecture.

The literature selection is primarily focused on recent academic sources within the last 10–15 years to align with modern architectural practices. Peer-reviewed research is strictly required to support central claims regarding SOA, monolithic architectures, and the microservice granularity problem, ensuring that the theoretical foundation relies on validated scientific evidence rather than unverified commercial material.

In some cases, a limited use of wikipedia, and other traceable and reputable internet sources were used, such as GeeksforGeeks, Amazon Web Services, only for basic definitions of technical terms to improve readability and accessibility when introducing domain-specific

concepts. However, all such references are carefully verified to maintain academic credibility.

3.2 Experimental Environment and System Setup

All experiments were conducted in a controlled local development environment to ensure consistency and repeatability of results. The entire microservice-based system was deployed on a single local machine running Windows, where each service was executed independently in separate terminal instances.

All implementation activities were conducted by the author and the links to the related repositories are attached to this study and accessible through the navigation of Attachment B.

The technologies used include, but are not limited to, React, Express, Node.js, and MongoDB. This combination represents an industry-standard technological stack widely selected for modern distributed systems and microservices development due to its asynchronous performance characteristics [17]. The repositories are maintained with simple documentation of each service, and the relevant code repositories attached (see Attachment B).

The backend system was implemented using Node.js and Express, with each microservice (including the API Gateway, Authentication Service, User Service, and Property Service) running as an independent Node.js process. Communication between services was handled through RESTful APIs routed via the API Gateway service.

The database layer was implemented using MongoDB Atlas, a cloud-based database service, following a database-per-service pattern. Each microservice maintained its own isolated database within a shared MongoDB Atlas cluster, ensuring logical separation while preserving service independence and scalability.

To ensure consistency across experiments, identical datasets were seeded into the databases. These datasets were derived from a structured JSON dataset originally defined in the e-property-service GitHub repository (see Attachment B). This allowed direct comparison between static JSON-based responses and database-driven API responses under controlled conditions.

For frontend-related experiments, the React-based web client was executed locally and interacted with backend services through HTTP requests routed via the API Gateway.

Network-level measurements for data access experiments were captured using Wireshark, while frontend rendering performance was measured using browser developer tools across Google Chrome, Microsoft Edge, and Mozilla Firefox.

Load testing was performed using a custom Node.js script located in the e-property-service GitHub repository (see Attachment B). The script simulated concurrent user requests to the property listing endpoint through the API Gateway in order to evaluate system scalability under controlled and repeatable conditions. The script generated multiple simultaneous HTTP requests to emulate increasing user load, enabling measurement of response time behavior under stress conditions.

Logging middleware was implemented in both the API Gateway and Authentication Service to capture request timing, payload sizes, and response metadata. This ensured consistent measurement across authentication experiments.

The described environment provides the technical foundation required to execute the experiments consistently across all evaluation scenarios.

3.3 Experimental Procedure

Based on the established environment, a standardized experimental procedure across the implemented microservice architecture was applied to ensure repeatability, traceability and consistency of data collection.

Each experiment involved starting the required backend services independently in separate terminal instances, including the API Gateway, Authentication Service, User Service, and Property Service. All services communicated through RESTful APIs routed via the API Gateway.

For experiments involving authentication, custom logging middleware was implemented within both the API Gateway and Authentication Service. This middleware recorded request timestamps, payload sizes, and response durations. The middleware was integrated directly into the Express request pipeline to ensure that all incoming and outgoing requests were captured consistently. During execution, both local authentication and OAuth 2.0 flows were tested under identical conditions, and logs were collected from both services for comparison.

For the data access experiments, two separate implementations were executed: a static JSON response module and a MongoDB-based API endpoint. In both cases, property data was accessed through the React web client, which triggered HTTP requests routed via the API Gateway. Network traffic was captured using Wireshark to measure packet-level differences, data transfer volume, and request-response behavior. Both implementations were executed using an identical dataset to ensure consistency and enable direct comparison of performance and network overhead between static and dynamic data retrieval approaches.

For frontend rendering experiments, the React client was executed locally and connected to backend services through the API Gateway. Browser developer tools were used to measure rendering time and network loading behavior across multiple browsers. Dataset sizes were systematically varied while keeping all other conditions constant.

For scalability testing, custom load-testing scripts available in the GitHub repository of e-property-service, accessible in attachment B were executed to simulate concurrent users. These scripts sent simultaneous requests to the property listing endpoint through the API Gateway. Response times and system behavior were recorded for each test run under increasing load conditions.

To improve internal validity, all experiments were conducted under controlled conditions using the same hardware environment, identical datasets, and consistent software configurations. Only the independent variable under investigation was modified between experimental runs, while all other system parameters remained unchanged. This approach reduces the influence of external factors and improves the reliability and comparability of the collected results.

The procedure described above was applied throughout the evaluation process. The following section presents the individual quantitative experiments and the metrics collected for each evaluation scenario.

3.4 Experiments

To ensure traceability between the research questions, the implemented MVP, and the conducted experiments, the experimental design is structured around a clear mapping between architectural components and evaluation scenarios. This ensures that each experiment directly evaluates a specific part of the proposed microservice architecture and contributes to answering the corresponding research questions.

The following table presents the relationship between research questions, MVP components, techniques and the conducted experiments.

Table 2: Mapping of Research Questions to Experiments and MVP Components

Research Question	**MVP Components**	**Techniques**
RQ1: How can a microservice architecture be designed to support the core functionality of a fractional real estate investment platform?	API Gateway, Auth Service, User Service, Property Service	Architectural design + service decomposition (Chapter 4)
RQ2: How does architectural data and service layer design affect modularity, maintainability, and performance?	Auth Service, Property Service, Web Client	Experiment 1 (Authentication), Experiment 2 (Data access), Experiment 3 (Rendering)
RQ3: How well does the proposed architecture support scalability and system evolution when evaluated using defined architectural scenarios such as increasing numbers of users and property listings ?	API Gateway + backend services	Experiment 4 (Load testing / scalability)

Each experiment is designed to evaluate specific architectural decisions within the proposed microservice system. The experiments are quantitative in nature and focus on measuring performance, scalability, and architectural trade-offs under controlled conditions. Together, they provide empirical evidence for analyzing how design decisions in authentication, data access, client-side rendering, and system scalability impact the overall system behaviour.

Each quantitative experiment follows a controlled quantitative design where identical system conditions are maintained across test runs to ensure comparability. The experiments focus on measuring performance-related metrics derived from architectural decisions implemented in the MVP. The results are later analysed in Chapter 5 in relation to the defined research questions.

1. **Quantitative Experiment 1- Authentication Architecture: Local Auth vs OAuth2.0**

   This experiment evaluates how different authentication mechanisms impact system performance, security, and architectural modularity in relation to RQ2. The experiment is conducted on the Authentication Service and API Gateway components of the MVP architecture.

   Two authentication flows are implemented: a local authentication system using JWT-based credential validation and a third-party OAuth 2.0 authentication flow using Google as an identity provider. Both authentication mechanisms are executed through the API Gateway, and middleware logging is used in both the API Gateway and authentication service to capture runtime metrics.

   Each authentication flow is executed multiple times under identical conditions. The collected data includes average response time (ms), data transfer size (KB), and qualitative security considerations. The results are compared to identify performance and architectural trade-offs between local and delegated authentication approaches.

2. **Quantitative Experiment 2 - Data Access Layer: Static JSON vs API/Database:**

   This experiment evaluates the performance and network overhead differences between static JSON-based data access and dynamic API-based database retrieval. The experiment is directly linked to RQ2 and focuses on the Property Service component of the MVP.

   Two data access strategies are implemented: static JSON responses and database-backed API responses using MongoDB. Network traffic is monitored using Wireshark to capture packet-level differences between the two approaches.

   Metrics collected include packet count, data volume (KB), and average response time (ms). The objective is to evaluate the trade-off between performance efficiency and maintainability in dynamic systems.

3. **Quantitative Experiment 3 - Property Listing Rendering on Client side:**

   This experiment evaluates how architectural decisions in the backend influence client-side performance and user experience. It is linked to RQ2 and focuses on the Web Client component of the MVP.

   Rendering performance is measured across three dataset sizes (small, medium, large) using three different browsers. Browser developer tools are used to capture rendering time and network loading behaviour.

   The collected metrics include rendering time (ms) per dataset size. The experiment aims to evaluate how system architecture affects frontend responsiveness under increasing data complexity.

4. **Quantitative Experiment 4 - Load Testing of Simulated users querying property listings:**

   This experiment evaluates system scalability under increasing concurrent user load, directly addressing RQ3. The experiment involves the API Gateway and backend microservices of the MVP.

   Simulated users ranging from 5 to 100 concurrently send requests to the property listing endpoint. Response times and system behaviour are recorded under increasing load conditions.

   Metrics include average response time (ms), maximum response time (ms), and error rate (%). The objective is to evaluate how well the microservice architecture handles increased load and to identify scalability limitations under stress conditions.

Chapter 4

Architecture Design

4.1 Architectural Foundations

Modern distributed systems have proven to adopt microservice architecture as a response to the limitations of traditional monolithic systems [4][5]. A traditional software design approach, monolithic architecture comprises the user interface, business logic, data access layers developed in a waterfall approach and packaged into a single deployable unit. This means that any change to the system requires redeployment of the entire system. While this architectural approach is simpler to develop and deploy, it introduces scalability limitations and increases tight coupling between system components, thereby reducing flexibility and limiting future system and technological evolution. This limited architecture is illustrated in Figure 2 below, where all the system functionalities are tightly coupled and exist in a single unified deployable application.

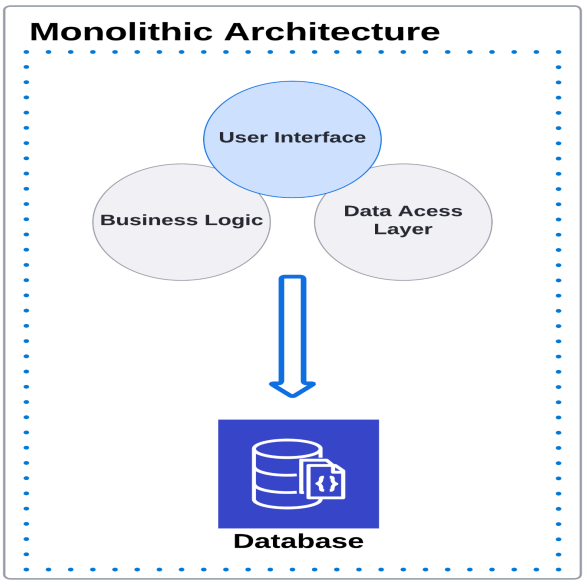


Figure 2: Overview of a Monolithic applications architecture

Service-Oriented Architecture (SOA) extends this concept by restructuring the application into several independent services that communicate over a network. This architecture introduced service decomposition and distributed system principles, allowing logically different components to operate independently towards a single flow of task in an application. However, SOA implementations often rely on a centralized middleware such as an Enterprise Service Bus (ESB) to manage the communication between services adding a

layer that introduces complexity, increased latency and reduced flexibility in a systems evolution [3].

Microservice architecture is considered an evolution of SOA, where dependency on centralized ESB is substituted by lightweight communication mechanisms such as REST APIs [3]. This allows the services to be distributed and less coupled to the central system and make them recognisable as independently deployable miniature applications, increasing scalability. This makes the microservices align with the modern practice of breaking down large system components into granular services that follow agile development, deployment and cloud-native application principles [5]. This architectural shift is illustrated in Figure 3 below, where the system is decomposed into granular independent services consisting of separate services, databases, and cloud solutions.

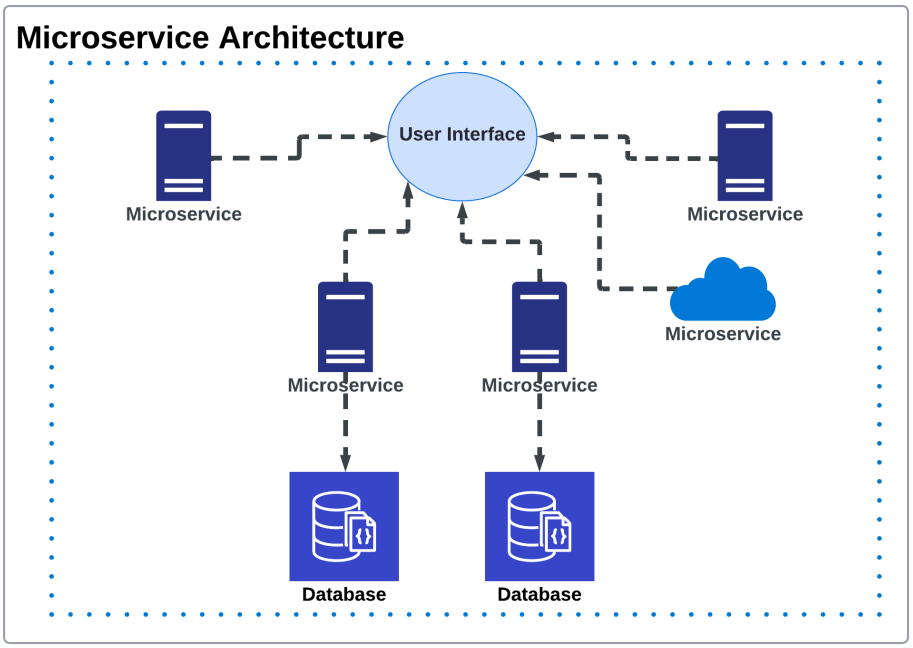


Figure 3: Overview of a Microservice architecture

According to Dragoni et al. [5], a microservice-based architectural design is structured as a collection of loosely coupled services, each responsible for a specific business function. This improves maintainability of codebase and service, enabling independent evolution of system components. Furthermore, research highlights that such systems provide improved scalability, positive impact on businesses and fault isolation as compared to traditional monolithic architectures [5].

The introduction of microservices and transitioning into this architecture is not easy. Hassan et al. [13] emphasized that correctly forming the granular services is one of the most critical challenges in microservice design as overly fine-grained services increase communication overhead, while coarse-grained services reduce the modularity benefits. Therefore a careful decomposition grounded in core business-required services is to be studied in order to design the optimal architectural solution.

In this study, a microservice-based architectural approach is selected, in relation to fractional real estate investment platform's requirements. The architecture is designed to support scalability, maintainability, performance efficiency, and the future evolution of the platform.

4.2 Platform Workflow Overview

The core workflow of a modern fractional real estate investment platform provides a high-level understanding of user interactions and system scope. The workflow is based on the principles of fractional ownership, where multiple investors can collectively acquire shares in real estate assets through a digital platform [1].

The platform supports three primary user roles: investors, property owners, and platform administrators. Investors can register and authenticate themselves through the platform, browse available property listings, review investment opportunities, and purchase fractional ownership shares in listed properties. Property owners can submit commercial properties to the platform for investment consideration, enabling capital to be raised from multiple investors. Platform administrators are responsible for reviewing submitted properties, managing platform operations, and maintaining the integrity of the platform.

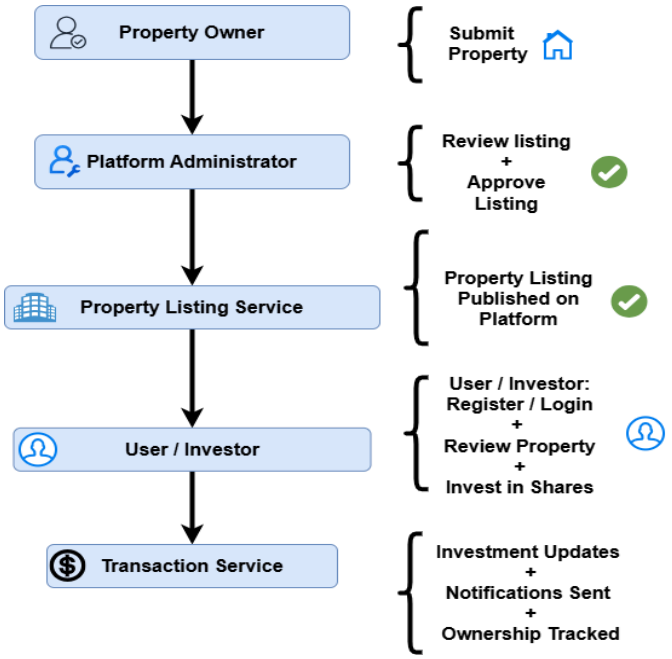


Figure 4: Overview of a platform workflow

Figure 4 presents a conceptual overview of the user interactions and business workflow that form the basis of the architectural design proposed in this study. Property owners submit properties for review, platform administrators review and approve listings, and investors can subsequently browse available properties and purchase fractional ownership shares. The resulting investments are recorded and can be made available through portfolio tracking and notification functionality.

This workflow defines the core functional boundaries of the system and serves as the foundation for the architectural decomposition presented in the following sections. In

particular, the separation of responsibilities across different user roles motivates a service-oriented design where authentication, property management, and investment-related functionality are handled as independent components.

4.3 Service decomposition

The fundamental step of designing a microservice architecture is service decomposition, where our fractional real estate application's core functionalities can be used in order to divide into distinct services based on individual capabilities [5] [6]. Each service is responsible for a specific domain, such as user authentication, user data storage for management, or property handling, investment service connecting the user to the properties, a notification service for keeping customers updated and finally a payment service. Each of these services operate independently with its own data storage.

As highlighted by Vu Nguyen Huynh Anh [6], in fintech systems, service decomposition must consider the business boundaries and security requirements as financial applications require strict isolation between sensitive building blocks, such as authentication and transaction processing. Ensuring that failure or vulnerabilities in a single service does not compromise the entire system.

In this thesis, the system is broken down into several core services adhering to avoidance of introducing the granularity problem into the solution [13], including authentication, user management, property management, property management, API Gateway coordination, Transaction management, payment service and notification service as shown in table 1**,** the table also states the isolated responsibilities for each service. and the scope for each service is presented clarifying the limited depth of this study and the value it brings to the experiments designed and described in chapter 3.

Table 3: Decomposition of backend to services

Service	Database	Responsibility	Scope
Auth Service	auth_db	authentication, registration, OAuth 2.0 login, password hashing, role assignment	Core MVP + Experimental
User Service	user_db	Manages user profiles, preferences, and references to investments	Core MVP + Experimental
Property Service	property_db	property listings (CRUD, filtering, retrieval)	Core MVP + Experimental
API Gateway Service	no database required/ uses service-associated databases indirectly	Internal aggregation via services	Core MVP
Transaction Service	transaction_db	Manages investment records linking users and properties	Future work

Messaging Service	queue or db	Handles notifications (in-app/email alerts)	Future work
Payment Service	no database required / Third party application	Handles financial transactions and payment processing	Future work

Additionally, Waseem et al . [12] argue that microservice-based systems require strong monitoring and observability mechanisms due to their distributed nature. This further justifies the inclusion of logging middleware in the architecture to track behavior across the data and communication layers of the system.

The decomposition table 3 shows the strategy adopted, taking into account the separation of concerns, improved scalability and isolated deployable capabilities, which are essential for a modular financial application.

4.4 Proposed System Architecture

The proposed architecture is designed as a distributed microservice ecosystem where each service operates independently which are reusable for multiple client applications such as a web application, android and apple mobile web applications. The communication is handled through a central API Gateway. This gateway acts as a single entry point for all client requests and routes them to the appropriate backend services, as illustrated in Figure 5, the architectural overview.

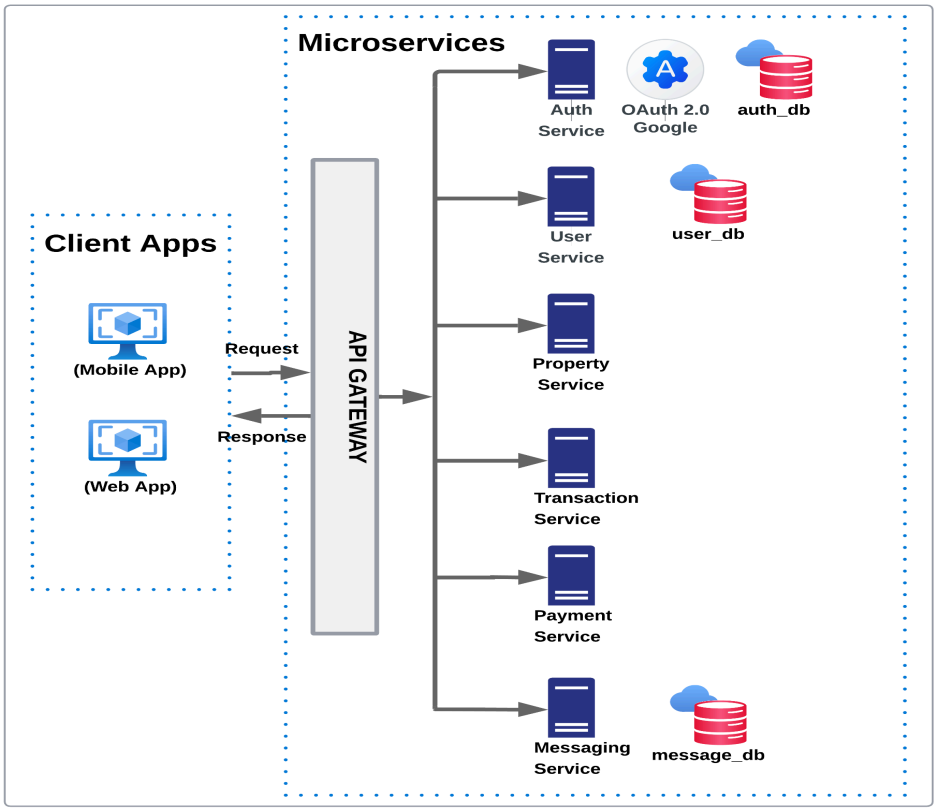


Figure 5: Overview of the proposed Microservice architecture.

Microservice architectures are commonly adopted because they facilitate independent service deployment and provide a foundation for future scalability and system evolution. According to Dragoni et al. [5], such architectures are suitable for systems that require frequent updates and independent deployment cycles, which aligns with the requirements of a modern financial platform.

However, scalability and performance must be carefully evaluated in such distributed systems. Rodrigues et al. [14] emphasizes that modularity introduced by microservices introduces overhead and latency due to their inter-service communication. This trade-off is a key consideration in the proposed architecture, especially in authentication flows and data access layers.
The system designed in this study follows the domain-driven decomposition model, ensuring that each service contains its own separate data, logic and database. This aligns with the findings from Huynh Anh [6], who highlights that fintech systems benefit from strict service boundaries to improve security and maintainability.

The deconstructed, services-based architecture, shown in figure 5**,** illustrates the decomposition of the system into core microservices that are required by such a financial application, while highlighting the role of API Gateway as the central communication layer between the clients and backend services.

The proposed architecture was developed iteratively based on continuous refinement of service decomposition and evaluation of architectural trade-offs identified through literature review and experimental design considerations. This iterative process allowed progressive improvement of service boundaries and system structure to better align with scalability, modularity, and fintech domain requirements.

Overall, the proposed architecture takes into account and balances modularity, scalability and performance trade-offs, making it suitable for a cloud-native fractional real estate investment platform.

4.5 Authentication Architecture Design

Authentication is a crucial component in financial applications, where secure authentication and authorization for a user is key in building trust for a platform [6]. In this study, two authentication approaches are implemented and evaluated within the proposed microservice architecture. A traditional local authentication and third-party OAuth 2.0 authentication using Google as an identity provider.
The two ways of authentication:

1. OAuth modern, secure, leveraging external security assurances
2. Local auth , database dependent, vulnerable to security threats

The first approach, a local authentication architecture, illustrated in Figure 6, where users provide credentials directly to the application. The client communicates through the API Gateway to the authentication service, which then validates credentials stored in an isolated database and generates a JSON Web Token (JWT) for the authenticated access and provides back to the user’s client through the API Gateway.

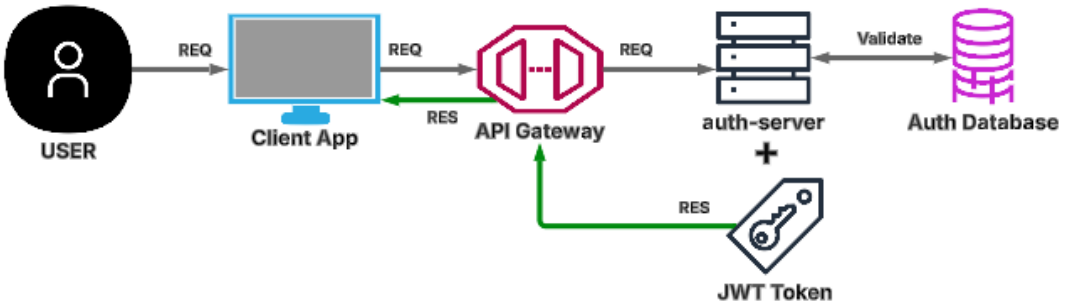


Figure 6: Overview of the Local Authentication Architecture and Flow Diagram.

This authentication method provides full control to the application over the management and authentication of the user. However it also increases responsibility of the system regarding data storage, credential validation and security management. As highlighted by Huynh Anh [6], security boundaries are important in fintech systems due to sensitivity of user-related data, this method of isolating authentication and authorization service is a key example in our separation of concerns based design of the proposed architecture.

The second approach, a Google OAuth 2.0 Cloud Service, illustrated in Figure 7, where authentication is handled though an external third-party cloud service provider. In this flow the user is redirected to Google for identification and verification before being returned to the application through a callback mechanism from the authentication service.

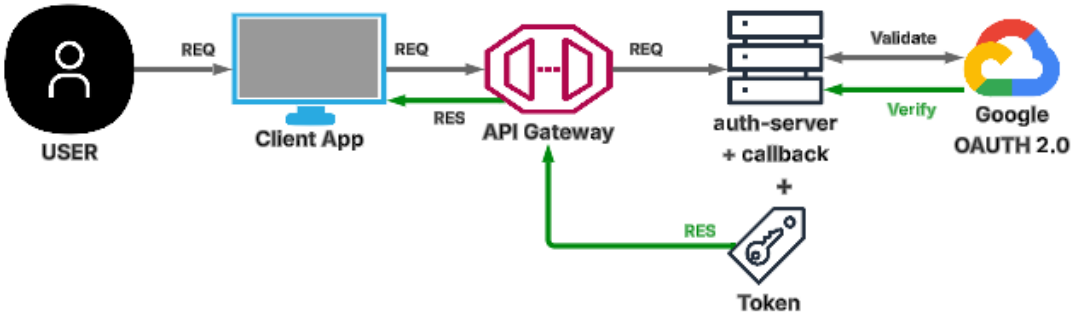


Figure 7: Overview of OAuth 2.0 Authentication Architecture and Flow Diagram

As compared to the local authentication method, OAuth 2.0 reduces direct credential handling, data storage and validation. This service improves trust through delegated authentication leveraging the credibility and secure architecture through the third-party cloud service. However, this method introduces additional steps in communication flow and dependency on external services, increasing architectural complexity and latency. According to Dragoni et al. [5], distributed systems often involve communication trade-offs that affect the performance and maintainability in order to increase architectural design efficiency.

The implementation of both authentication methods supports the experimental evaluation of authentication service, as described in Chapter 3. The experiment defined, enables the validation of choice of separation of authentication from the system, as a separate service which impacts scalability, modularity and security within the proposed microservice-based fintech platform architecture.

4.6 Data Architecture Design

The data architecture in a microservice-based system plays a crucial role in ensuring stability, scalability, modularity and service independence. In this study, a database-per-service pattern is adopted, where each service maintains its own isolated database [5] [6]. This approach avoids tight coupling that used to be in monolithic architectures, this ensures no service directly impacts the other services.

According to Huynh Nguyen Anh [6], fintech systems benefit largely through data isolation, as it improves the security boundaries and reduces the risk for cross-service data leakage and breaches. This is a significant factor in the viability and life of a financial application where sensitive user and transactional data must be strictly encapsulated within service boundaries.

In the proposed architecture and its minimal implementation, structured data such as user profiles, property listings and authentication records are stored in MongoDB databases assigned per service. This allows evolution of each independent service while maintaining control over its own data model. This architecture is not restricted to a single type of database and allows the introduction and usage of other database technologies allowing scalability without being restricted to a single database technology.

In addition to persistent storage, lightweight JSON-based data exchange is used between storages and clients through REST APIs. This ensures efficient communication by adopting a hybrid solution, while maintaining consistent data format across the system.

However, static JSON-based responses are primarily used for experimental comparison in Chapter 5, where performance differences between static and dynamic data retrieval are evaluated. This hybrid data approach where json API communication and database-per-service is used, ensures flexibility and scalability while enabling controlled experimentation for architectural evaluation.

4.7 Communication Architecture

The proposed architecture's communication architectural design comprises primarily the usage of the API Gateway pattern, this service acts as the central entry point for all client requests. All communication between the frontend and backend microservices is routed through this gateway, which handles request forwarding, authentication, and basic aggregation of response from multiple services.

According to Dragoni et al. [5], API Gateways are a common pattern in microservices as they provide a central interface while preserving service independence, and improving system maintainability and reduced direct coupling between clients and backend services.

In addition to internal service communication, as illustrated in figure 6 and figure 7, the system also interacts with external third-party services when necessary. A key example is the integration of the authentication service with Google OAuth 2.0, which is used for increasing security during the login process. This communication is limited to authentication flows and does not extend into core business logic but still takes into account the security and trust required from the customers from a financial application

The proposed architecture's implementation focuses on synchronous REST API communication to maintain simplicity and in the experimental setup and leave room for scalability and adoption of asynchronous communication patterns, which can be utilized in future improvement of the platform [5].

Overall, the communication design is limited to a simplified microservice interaction, where the API Gateway serves as the primary coordination layer between client, backend and third-party services.

Chapter 5

Results and Evaluation

5.1 Experiment 1 - Authentication Architecture: Local Auth vs OAuth2.0

The experiment compared local authentication and Google OAuth 2.0 authentication flows implemented within the proposed architecture. Six repeated trials were carried out for both authentication methods and an average was taken for the recorded values. Logging middleware attached to the API Gateway and authentication service recorded response times, request sizes and response times.

Table 4 presents the measured performance results focusing on runtime efficiency and network payload size.

Table 4: Authentication Method Comparison Results

Metric	Local Auth	OAuth2.0 (Google)
Avg Response Time (ms)	138	266
Data Transfer (KB)	approx. 0.3 KB	approx. 0.35 KB

Table 5 presents the qualitative, author-based assessment of each authentication method in terms of architectural qualities such as security and scalability. This table complements the performance metrics in Table 4 by evaluating non-quantifiable design aspects.

Table 5: Authentication Method Author-based assessment

Metric	Local Auth	OAuth2.0 (Google)
Complexity (qualitative)	Low	Medium/High
Security level	Medium	High
Modularity and scalability	Medium	High

The results in Table 4 show that local authentication achieves lower average response time (138 ms) compared to OAuth 2.0 (266 ms). Table 5 provides the corresponding architectural interpretation, where OAuth 2.0 is assessed as more secure and scalable but also more complex to implement. This separation allows performance and architectural quality to be evaluated independently.

5.2 Experiment 2- Data Access: Static JSON vs API and Database

The experiment compared the static JSON-based data access with dynamic API-based database retrieval for the property listings. Five repeated test runs were performed under identical conditions, and response times were recorded.

Table 6 presents the raw response time measurements for both approaches across all test runs.

Table 6: Raw Response Time Results

Test Run	Static JSON API (ms)	MongoDB API (ms)
1	12.50	80.00
2	18.80	64.70
3	26.00	73.00
4	21.60	78.10
5	19.50	68.90

To provide a clearer comparison beyond raw test runs, the aggregated results are summarized in Table 7. This table combines response time, packet count, and data volume to highlight overall system behavior at a network level.

Table 7: Static JSON vs API/Database Comparison

Metric	Static JSON	API/Database
Packet Count	9	9
Data Volume (KB)	1.5	1.3
Avg Response Time (ms)	19.68	72.94

Table 8 extends the evaluation by presenting an author-based assessment of maintainability, scalability, and dynamic update capability. This complements the technical metrics in Table 7 by focusing on long-term architectural implications.

Table 8: Static JSON vs API/Database Comparison Author-based Assessment

Metric	Static JSON	API/Database
Maintainability	Low	High
Scalability	Low	High
Network Overhead	Lower	Higher
Dynamic Updates	No	Yes

As shown in Table 7, static JSON provides lower response times due to the absence of database queries and reduced processing overhead. However, API-based access introduces additional latency while improving system maintainability, scalability, and data consistency, as reflected in Table 8.

Wireshark analysis (Appendix A) further supports these findings by showing that both approaches generate identical packet counts, while API-based access results in slightly lower data volume due to structured query responses. This indicates that performance differences are primarily caused by processing and database overhead rather than network packet frequency.

5.3 Experiment 3 - Frontend Rendering Performance

This experiment evaluates client-side rendering performance under different dataset sizes across multiple browsers.

The results are presented in Table 9, which shows rendering time variations across dataset sizes.

Table 9: Rendering Performance Results

Dataset Size	Browser 1 (ms)	Browser 2 (ms)	Browser 3 (ms)
Small (60 listings)	12.00	8.00	11.60
Medium (90 listings)	21.10	16.50	18.50
Large (1080 listings)	37.10	29.00	35.00

As illustrated in Table 9, rendering time increases proportionally with dataset size, indicating that client-side performance is directly impacted by data volume.

This trend is further visualized in Figure 8, which shows a steady increase in rendering time as dataset size grows. Minor performance differences are observed across browsers, specifically Google Chrome, Microsoft Edge, and Mozilla Firefox, but the overall trend remains consistent across all test cases.

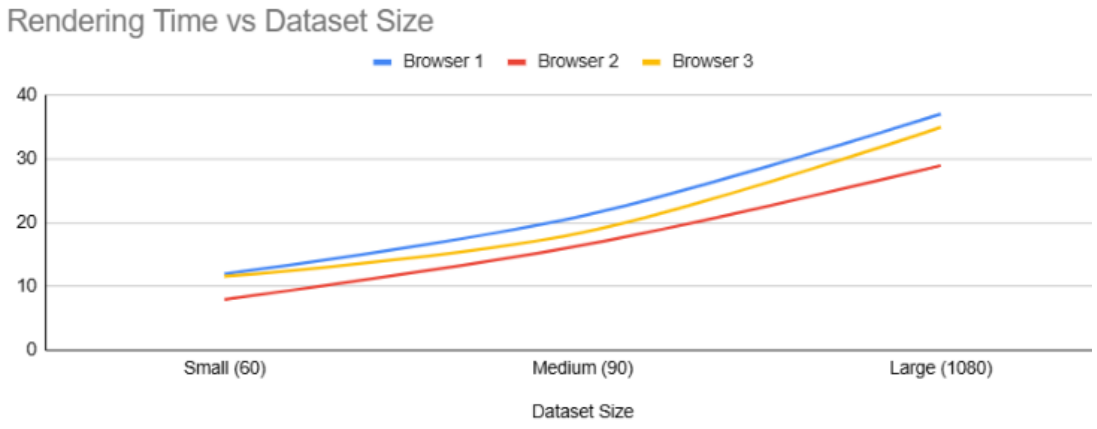


Figure 8: Rendering Time (ms) VS Dataset size line graph

In Figure 8, Rendering Time (ms) VS Dataset size line graph shows increase in render time with increased dataset size.

5.4 Experiment 4 – Load Testing Results

This experiment evaluates system scalability under concurrent user load. Simulated users (5–100) generated simultaneous API requests for property listings.

The results are shown in Table 10, which presents system performance under increasing load.

Table 10: System Load Test Results

Concurrent Users	Avg Response Time 5x (ms)	Max Response Time (ms)	Error Rate (%)
5	63.1	148	0
10	73.53	213.6	0
20	86.6	149.6	0
100	325.7	549.7	0

As shown in Table 10, system performance remains stable under low to medium load conditions (5–15 users). However, at 20-100 concurrent users, response times increase significantly, indicating emerging scalability limitations under higher load. The error rate could not be captured due to the experiment's local environment and no failures in the communication layer were detected during the testing. This trend is visualized in Figure 9, which shows a sharp increase in average response time as concurrent users increase.

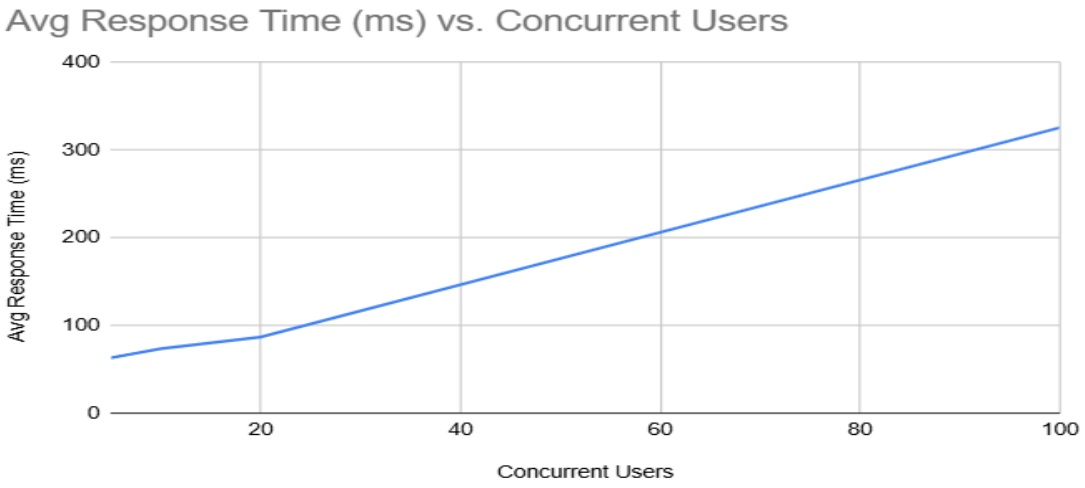


Figure 9: Average response time (ms) Vs Concurrent Users Graph

Chapter 6

Analysis and Discussion

This chapter presents an analysis of the experimental results in relation to the proposed microservice-based architecture for the fractional real estate investment platform. The findings from the conducted experiments are interpreted using the design science research approach, where the system is first designed, then evaluated through controlled experiments, and finally analyzed to derive architectural insights. The discussion focuses on how different architectural decisions, including authentication mechanisms, data access strategies, frontend rendering, and system scalability under load, influence performance, modularity, maintainability, and scalability. Based on these evaluations, the following sections address each research question individually.

6.1 Research Question 1

How can a microservice architecture be designed to support the core functionality of a fractional real estate investment platform?

Research Question 1 is addressed through the architectural design and implementation process presented in Chapter 4, rather than through experimental evaluation. As outlined in Table 2 (*Mapping of Research Questions to Experiments and MVP Components*), RQ1 is primarily linked to the system design activities, including workflow analysis, service decomposition, and the definition of the overall microservice architecture.

The design process began with an analysis of the platform's business domain and stakeholder interactions. As illustrated in Figure 4 (*Overview of a Platform Workflow*), the system consists of three primary stakeholders: investors, property owners, and platform administrators. These actors interact through core platform functions such as authentication, property browsing, property submission, investment actions, and administrative review. This workflow analysis, presented in section 4.2 (*Platform Workflow Overview*), provided the foundation for identifying the system's core functional requirements.

Based on this workflow, the system was decomposed into independent services according to business capabilities, as described in Section 4.3 (*Service Decomposition*). This decomposition approach is aligned with microservice architectural principles, where system functionality is structured around domain-driven boundaries rather than technical layers. Dragoni et al. emphasize that microservices should be designed as independently deployable units organized around specific business capabilities, enabling modular system evolution and reducing tight coupling between components [5]. However, research highlights that defining appropriate service boundaries is one of the most challenging aspects of microservice adoption, particularly due to the "granularity problem," where services can become either too fine-grained or too coarse-grained [13]. In this design, the selected services—authentication, user management, and property management, represent a balanced granularity level intended to maintain both modularity and manageable inter-service communication. This decomposition therefore reflects a domain-oriented approach that aligns service boundaries with core business responsibilities identified in Section 4.2.

The resulting architecture, presented in Section 4.4 (*Proposed System Architecture*), consists of an API Gateway, Authentication Service, User Service, and Property Service. The API Gateway functions as the single entry point for client requests and is responsible for routing and request coordination across backend services. The Authentication Service handles user authentication and authorization processes, while the User Service manages user-related data and operations. The Property Service is responsible for handling property listings and related business logic. This service structure directly reflects the decomposition strategy defined in Section 4.3 and the workflow requirements identified in Section 4.2.

The MVP implementation indicates that the proposed architecture is capable of supporting the core functional requirements of the platform. The independently deployable services were able to collectively support authentication, user management, and property listing functionality through well-defined interfaces. This confirms that the service decomposition strategy derived from Section 4.3 is not only theoretical but also practically implementable within the MVP system.

Furthermore, the architecture provides a structured foundation for future system expansion. As the platform evolves, additional services such as payment processing, transaction management, messaging, and portfolio tracking can be introduced without modifying the existing services. This extensibility is a direct outcome of the modular design approach used during service decomposition and system architecture definition.

In addition, the design process demonstrates an iterative relationship between understanding the domain and refining system structure. As requirements were further analyzed, service boundaries were adjusted to better reflect functional responsibilities. This indicates that the architecture is not static but evolves alongside the refinement of domain understanding, which is consistent with best practices in microservice-based system design.

Therefore, Research Question 1 is answered through the end-to-end design process presented in Chapter 4, including workflow analysis (Section 4.2), service decomposition (Section 4.3), and system architecture definition (Section 4.4). The design and MVP implementation indicate that a microservice architecture organized around business capabilities can effectively support the core functionality of a fractional real estate investment platform while maintaining modularity, extensibility, and alignment with domain-driven design principles.

6.2 Research Question 2

How does architectural data and service layer design affect modularity, maintainability, and performance?

The experiments show that architectural decisions in the data and service layers have a direct impact on performance, maintainability, and modularity. Across the evaluated scenarios, a consistent trade-off emerges between system efficiency and architectural flexibility, which is a well-established characteristic of microservice-based systems.

The authentication experiment (Section 5.1, Table 4 and Table 5) shows that local authentication achieves lower average response times (138 ms) compared to OAuth 2.0 (266 ms). However, this comparison must be interpreted in the context of system boundaries. The OAuth 2.0 flow includes communication with an external identity provider (Google),

meaning that the measured response time includes external network latency and third-party processing outside the system architecture. In contrast, local authentication operates entirely within the system boundary. Therefore, the difference observed in Table 4 reflects both architectural design and external dependency overhead, rather than a strictly internal system comparison.

This aligns with Ravikumar [18], who highlights that OAuth-based authentication improves security delegation and reduces credential management responsibility but introduces additional communication overhead and latency due to external authorization flows. The qualitative evaluation in Table 5 (*Authentication Method Author-based assessment*) represents author-based assessment rather than direct experimental measurement, and is included to support interpretation of architectural trade-offs rather than to serve as empirical evidence.

The data access experiment (Section 5.2, Table 6–Table 8) compares static JSON-based responses with API/database-driven retrieval. Static JSON achieves lower response times (19.68 ms) compared to API/database access (72.94 ms). However, this difference is partially confounded, as API-based access includes both database query execution and network communication through the API Gateway, while static JSON represents locally served or in-memory data retrieval. As a result, the measured performance gap reflects combined network and database overhead rather than isolated contributions from each layer.

This observation is consistent with Dragoni et al. [5], who emphasize that microservice architectures inherently introduce communication overhead due to distributed service interaction, but provide improved modularity and independent deployment capabilities. In this study, the higher latency observed in Table 7 (*Static JSON vs API/Database Comparison*) is a direct reflection of this architectural trade-off between performance efficiency and system modularity.

In addition, the network analysis shows that both approaches generate the same packet count, while the API/database approach results in slightly lower overall data volume (KB) due to structured query responses compared to static payload transfer. This indicates that the performance difference is not driven by packet frequency, but rather by differences in payload processing and backend query execution.

Furthermore, Huynh Anh [6] highlights that fintech systems require strict service decomposition and data isolation to ensure maintainability and security of sensitive financial data. The author-based assessment in Table 8 (*Static JSON vs API/Database Comparison Author-based Assessment*) supports this argument, as the API/database approach improves maintainability, scalability, and dynamic update capability, despite introducing additional latency compared to static data access [5] [18].

The frontend rendering experiment (Section 5.3, Table 9 and Figure 8) shows that rendering time increases proportionally with dataset size. As illustrated in Figure 8 (*Rendering Time (ms) VS Dataset size line graph*), larger datasets significantly increase rendering time across the three tested browsers. This indicates that backend architectural decisions (data volume and API structure) directly influence frontend performance, reinforcing the importance of end-to-end architectural design rather than isolated backend optimization.

Finally, the scalability experiment (Section 5.4, Table 10 and Figure 9) demonstrates that system response time increases under higher concurrent load. As shown in Figure 9 (*Average response time (ms) vs concurrent users*), performance degradation is gradual at lower loads but becomes more pronounced between 20 and 100 concurrent users. This behavior is expected in microservice systems due to distributed request routing through the API Gateway and multiple service interactions.

Overall, when interpreting Tables 4–10 and Figures 8–9, a consistent architectural pattern emerges. Lightweight approaches such as static data access and local authentication provide better raw performance but reduce flexibility and security delegation. In contrast, distributed approaches such as API/database integration and OAuth 2.0 introduce additional communication overhead due to external dependencies and service decomposition but improve modularity, maintainability, and security.

While these trade-offs negatively affect raw performance metrics, they are acceptable in the context of financial systems, where security, data isolation, and system structure are more critical than minimal latency. As highlighted by Huynh Anh **[6]**, fintech architectures prioritize strict service boundaries and maintainability to ensure long-term system reliability and secure handling of sensitive financial data. Therefore, the observed performance overhead is justified by the increased architectural robustness and security guarantees provided by the microservice-based design.

Therefore, Research Question 2 is answered through the experimental evaluation presented in Chapter 5. The results and author-based assessments suggest that architectural decisions in the data and service layers significantly influence system behavior, and that microservice-based design inherently involves trade-offs between performance efficiency and long-term architectural qualities such as modularity and maintainability.

6.3 Research Question 3

How well does the proposed architecture support scalability and system evolution when evaluated using defined architectural scenarios such as increasing numbers of users and property listings ?

The scalability experiment (Section 5.4, Table 10 and Figure 9) evaluates how the proposed microservice architecture behaves under increasing concurrent user load. The system was tested using simulated users ranging from 5 to 100 concurrent requests targeting the property listing endpoint through the API Gateway, within a controlled single-machine environment described in Section 3.2.

The results in Table 10 (*System Load Test Results*) indicate a two-phase scalability pattern rather than a strictly linear or purely non-linear behavior. Between 5 and approximately 20 concurrent users, response time increases more sharply relative to load increments, indicating early system sensitivity to concurrency. This phase is likely influenced by request initialization overhead, warm-up effects in Node.js services, and early formation of request queues within the API Gateway and backend services. In contrast, beyond this range, the system transitions into a more stable degradation phase where response time increases more proportionally with load, as illustrated in Figure 9 (*Average response time (ms) Vs Concurrent Users Graph*).

This transition suggests that the system approaches a more stable processing state, where initial variability caused by cold-start execution and scheduling stabilizes, and performance becomes dominated by steady-state factors such as CPU scheduling and request throughput limits.

From an architectural perspective, this behavior can be explained by the distributed nature of the system. Each request passing through the API Gateway may trigger multiple service calls and database interactions, introducing cumulative latency. This is consistent with Dragoni et al. [5], who argue that microservice decomposition introduces inherent communication overhead due to distributed execution chains.

However, the observed scaling behavior is also strongly influenced by the deployment configuration. Since all services run as independent Node.js processes on a single machine, increased concurrency leads to resource contention at the operating system level. CPU scheduling, memory pressure, and event-loop contention in Node.js likely contribute to the increased response times observed after 20 concurrent users.

Importantly, despite the performance degradation, the system maintained zero error rate across all load levels, indicating that the architecture preserves functional stability even under stress conditions. This suggests that the primary limitation is not correctness or failure handling, but throughput efficiency.

It is also critical to interpret these findings as vertical scalability behavior under constrained deployment conditions, rather than horizontal scalability in a distributed production environment. In a horizontally scaled deployment, load balancing and service replication could reduce the bottlenecks observed here, particularly at the API Gateway and database interaction layers.

From a design perspective, these results reinforce the trade-off identified in Research Question 2, where modular microservice decomposition improves system structure and maintainability but introduces additional communication and coordination overhead. In the context of the scalability experiment, this trade-off becomes more pronounced under increasing load, indicating that performance degradation is not an isolated phenomenon but directly linked to the distributed nature of the architecture.

This interpretation is supported by Huynh Anh [6], who argues that microservice-based fintech systems prioritize modularity, service isolation, and long-term maintainability over raw execution performance. In this context, performance overhead is an expected consequence of achieving stronger architectural separation and system extensibility.

Overall, when interpreting Table 10 and Figure 9, the results show that the proposed architecture handles low to moderate workloads effectively but experiences expected performance degradation under higher concurrency. This behavior can be explained by the combination of API Gateway routing overhead, distributed service communication, and shared resource constraints within the deployment environment.

Therefore, Research Question 3 is answered through the scalability experiment in Section 5.4. The results demonstrate that the architecture maintains stability under load while exhibiting performance degradation consistent with known characteristics of distributed

microservice systems, but would likely require further optimization such as caching, asynchronous processing, or load balancing to support production-level scalability.

6.4 Overall Discussion

The common pattern observed across all experiments is that architectural improvements in modularity, maintainability, security, and flexibility often introduce additional performance overhead. The findings therefore suggest that architectural design should be evaluated as a balance between quality attributes rather than optimization of a single metric. While monolithic or static approaches often provide lower latency, the microservice-based alternatives provide greater adaptability and long-term maintainability, which are important characteristics for evolving fintech systems.

These findings indicate that the evaluated system properties are not independent, but interrelated, where improvements in one quality attribute directly influence others within the architectural design space.

Chapter 7

Conclusion

This study addressed the increasing need for scalable and modular fintech systems by exploring the limitations of traditional monolithic architectures in handling modern fractional real estate investment platforms. The goal of this work was to design and evaluate a microservice-based architecture capable of supporting a flexible and scalable system structure while enabling independent development and evaluation of core services.

A microservice-based system architecture was designed and implemented as an MVP, consisting of independently deployable services such as authentication, user management, and property handling, coordinated through an API Gateway. This design enabled separation of concerns, service isolation, and experimental evaluation of different architectural decisions across authentication, data access, frontend rendering, and system scalability.

The results from the conducted experiments show clear trade-offs between architectural choices. The evaluation suggests that the proposed microservice architecture improves modularity and scalability compared to a tightly coupled design, while introducing additional communication overhead between services. OAuth 2.0 improves security at the cost of higher latency, while static JSON improves performance but reduces flexibility compared to API-based database access. Load testing further indicates that the system performs well under low to medium load but shows performance degradation under higher concurrency levels.

The research questions are addressed through the design, implementation, and evaluation of the proposed MVP architecture. The experimental results demonstrate how architectural decomposition influences performance and scalability, while the architectural design provides insights into maintainability considerations within the evaluated scenarios. The findings highlight practical trade-offs associated with authentication mechanisms, data access strategies, frontend rendering performance, and service communication in a distributed system.

This study contributes a proposed microservice architecture for a fractional real estate investment platform together with an MVP implementation and an empirical evaluation of selected architectural decisions. The evaluation provides insights into the performance and scalability characteristics of the implemented system and illustrates how different design choices affect architectural quality attributes. The contribution extends existing research by providing an architectural and empirical perspective within the emerging domain of fractional real estate investment platforms.

The findings indicate that the proposed architecture provides a feasible foundation for supporting the core functionality of a fractional real estate investment platform while maintaining service separation and modularity. However, the evaluation was conducted using an MVP implementation in a controlled experimental environment. Therefore, the results

should be interpreted as evidence of feasibility and architectural behaviour within the evaluated scenarios rather than validation of a production-ready platform.

Future work may extend the architecture with additional business services and evaluate other quality attributes such as fault tolerance, security, monitoring, and large-scale deployment characteristics. Such investigations would provide a broader understanding of the long-term suitability of the architecture for real-world fintech environments.

Chapter 8

Future Work

There is significant potential for this study to be extended toward practical startup applications within the domain of fractional real estate investing. The current implementation focuses on core service decomposition that can be further extended to support additional business requirements and scalability needs. The proposed architecture can be further decomposed into additional microservices depending on domain complexity and evolving system requirements. Further work is also needed on the user interface to better align the platform with user needs and market expectations.

More work can be put into implementing transactional, messaging, and payment services, which would be required for a fully functional end-to-end platform. The API Gateway can also be extended to support additional client applications, such as mobile applications for Android and iOS.

A key limitation of this study is that all scalability experiments were conducted in a local development environment using single instances of each service. Future work should evaluate the proposed architecture in a production-like environment using containerization, orchestration, and multiple service instances. This would enable a more realistic assessment of horizontal scalability under distributed workloads.

In this context, the microservice-based architecture can be containerized and deployed using orchestration platforms such as Kubernetes, for example in cloud environments like Microsoft Azure. This would allow independent scaling, automated deployment, and management of multiple services in a cloud-native setup.

Although security is an important consideration within fintech systems, this study only evaluates authentication-related architectural decisions through the comparison of local authentication and OAuth 2.0. Future work should investigate additional security aspects such as user data protection, transaction integrity, payment security, audit logging, monitoring, and regulatory compliance. These areas would be essential before transforming the MVP into a production-ready fintech platform.

Additional future improvements include enhanced technical documentation, deployment of a public waitlist landing page, and further refinement of security mechanisms to ensure data privacy and transactional integrity.

The resulting architecture can serve as a foundation for startups seeking to develop scalable fintech platforms based on fractional ownership principles.

References

[1] “Fractional Ownership: Definition, Benefits, & Examples,”Investopedia. [Online]. Available: https://www.investopedia.com/terms/f/fractionalownership.asp#toc-what-is-fractional-ownership. [Accessed: Apr. 16, 2026].

[2] “Microservices,” Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Microservices. [Accessed: Feb. 6, 2026].

[3] “What’s the Difference Between SOA and Microservices?,” Amazon AWS. [Online]. Available: https://aws.amazon.com/compare/the-difference-between-soa-microservices/. [Accessed: Apr. 6,2026].

[4] “List of software architecture styles and patterns,” Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/List_of_software_architecture_styles_and_patterns. [Accessed: Feb. 6, 2026].

[5] N. Dragoni, et al., “Microservices: yesterday, today, and tomorrow,” *ResearchGate*. Jan. 2017. [Online]. Available: https://www.researchgate.net/publication/315664446_Microservices_yesterday_today_and_tomorrow [Accessed: Apr. 18, 2026].

[6] V. N. H. Anh, “An Architectural View Model for Designing and Implementing Microservices-based Systems: Use Case in FinTech,” *Procedia Computer Science*, vol 237, 2024. [Online]. Available: https://doi.org/10.1016/j.procs.2024.05.152 [Accessed: Apr. 15, 2026].

[7] “Introduction to Distributed System,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/what-is-a-distributed-system/ [Accessed: May 1, 2026].

[8] “Bibliotek,” Blekinge Tekniska Högskola. [Online]. Available: https://www.bth.se/bibliotek/. [Accessed: Feb. 1, 2026].

[9] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, Mar. 2004. [Online]. Available:

https://www.researchgate.net/publication/201168946_Design_Science_in_Information_Systems_Research. [Accessed: Apr. 16, 2026].

[10] “ResearchGate,” ResearchGate. [Online]. Available: https://www.researchgate.net/. [Accessed: Apr. 25, 2026].

[11] “Science Direct,” ScienceDirect. [Online]. Available: https://www.sciencedirect.com/. [Accessed: Apr. 20, 2026].

[12] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, Monitoring, and Testing of Microservices Systems: The Practitioners’ Perspective,” *Journal of Systems and Software*, vol. 182, 2021 [Online]. Available: https://doi.org/10.1016/j.jss.2021.111061. [Accessed: Mar. 7, 2026]

[13] S. Hassan, R. Bahsoon and R. Kazman, “Microservice Transition and its Granularity Problem: A Systematic Mapping Study,” arXiv:1903.11665v1,2019. [Online]. Available: https://arxiv.org/pdf/1903.11665 . [Accessed: Mar. 7,2026].

[14] H. Rodrigues, A. R. Silva and A. Avritzer “Assessment of performance and its scalability in microservice architectures: Systematic literature review,” *Journal of Systems and Software*, vol. 223, May 2025. [Online]. Available: https://doi.org/10.1016/j.jss.2025.112500. [Accessed: Feb. 25, 2026].

[15] “Google Scholar,” Google. [Online]. Available: https://scholar.google.com/. [Accessed: Apr. 3, 2026].

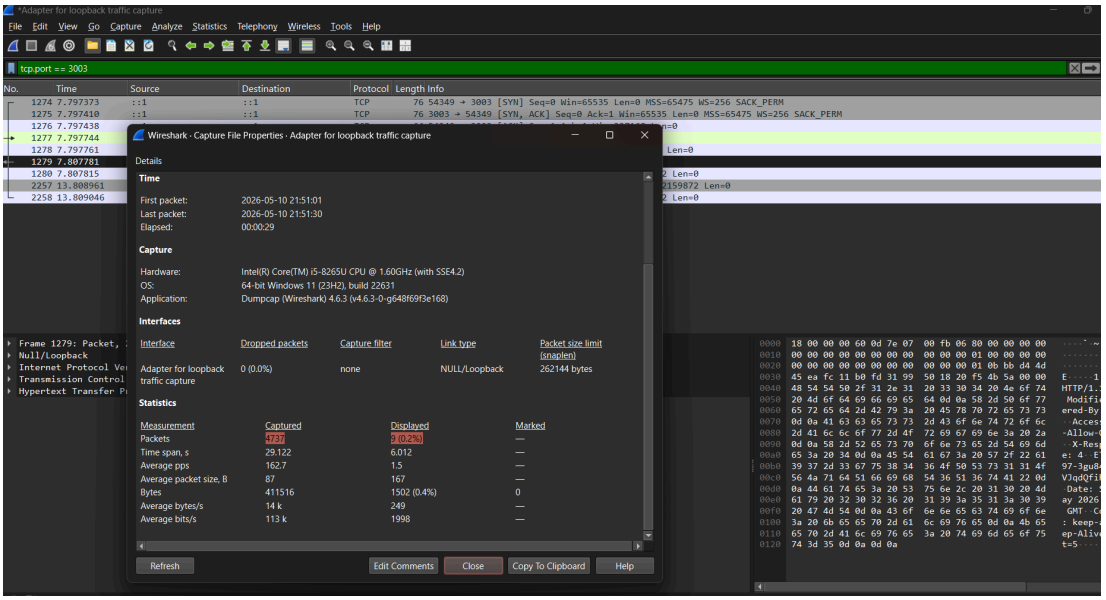
[16] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2008. [Online]. Available: https://doi.org/10.2753/MIS0742-1222240302. [Accessed: Apr. 16, 2026].

[17] Bhavyaa, S. Gupta and V. Vaishali, “Comprehensive Study of MERN Stack - Architecture, Popularity and Future Scope,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 7, no. 6, pp. 118–124, Dec. 2021, doi: 10.32628/CSEIT217630. [Online]. Available: https://www.researchgate.net/publication/357587510_Comprehensive_Study_of_MERN_Stack_-_Architecture_Popularity_and_Future_Scope . [Accessed: May 2, 2026].

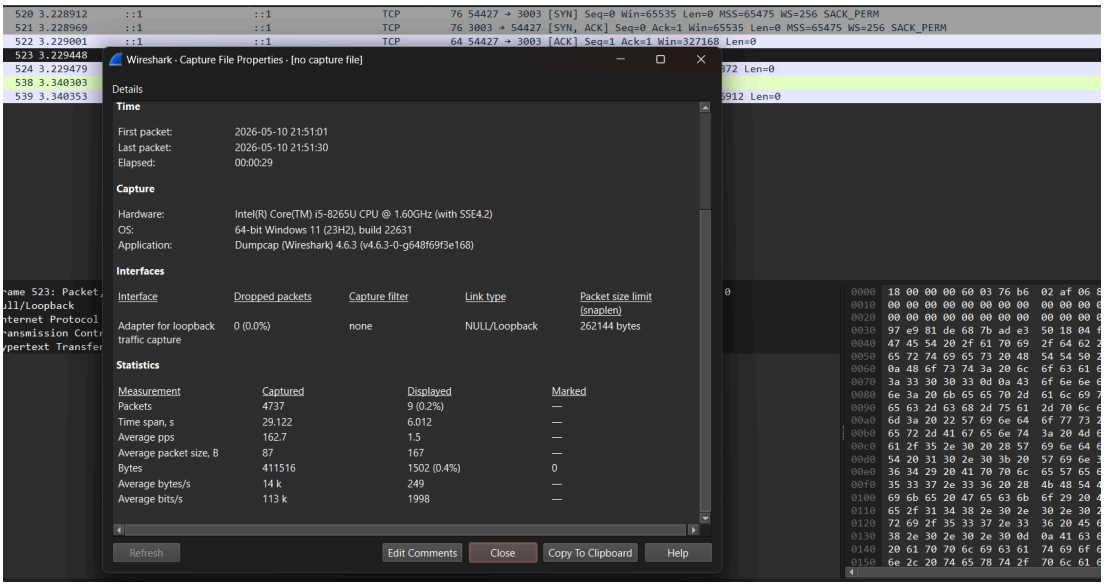
[18] S. Ravikumar, “OAuth 1.0 vs. OAuth 2.0: An In-Depth Analysis of Token Handling, Client Authentication, and Developer Usability,” *Journal of Mathematical & Computer Applications,* 2023. [Online]. Available: https://www.researchgate.net/publication/395115790_OAuth_10_vs_OAuth_20_An_In-Depth_Analysis_of_Token_Handling_Client_Authentication_and_Developer_Usability . [Accessed: May 1, 2026].

Attachment A - WireShark Results

A1: Static JSON API Result



A2: MongoDB API Result



Attachment B - GitHub repositories

Link to repos containing MVP Earnsome application solution

1. Earnsome Application api gateway + autherization:
   Link: https://github.com/hihassan1998/e-api-gateway-service

2. Earnsome Application Authentication server:+
   Link: https://github.com/hihassan1998/e-auth-server-service

3. Earnsome Application User Service:
   Link: https://github.com/hihassan1998/e-user-server-service

4. Earnsome Application Property Service:
   Link: https://github.com/hihassan1998/e-property-service

5. Earnsome Application Web Application: (User, Investor and Admin UI)
   Link: https://github.com/hihassan1998/e-web-client